LAB 06 — ATTENTION MECHANISM: REFLECTION SYNTHESIS

Tài Liệu Phân Tích & Thực Nghiệm Khoa Học

October 25, 2026

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng
Học kỳ: Học kỳ I - 2026
Chủ đề: From Recurrent State to Context-Aware Representation
Mạch kết nối: LAB 05 (Recurrent Models) ⟶ LAB 06 (Attention Mechanism) ⟶ LAB 07 (Transformer)

—

1. Tổng Kết Các Phát Hiện Thực Nghiệm Cốt Lõi

LAB 06 đánh dấu bước ngoặt tư duy phương pháp luận trong chuỗi bài tập NLP: từ việc ép buộc toàn bộ lịch sử chuỗi phải tuần tự đi qua một ”nút thắt cổ chai” trạng thái ẩn (hidden state h_T trong RNN/LSTM ở LAB 05) sang việc cho phép **tương tác trực tiếp không qua trung gian giữa bất kỳ cặp token nào** thông qua cơ chế Attention.

Các kết quả thực nghiệm nổi bật đã kiểm chứng các nguyên lý lý thuyết:

1. **Hiệu năng và Tốc độ hội tụ của Self-Attention**:

- Trên cùng tập dữ liệu mẫu chuẩn IMDb gồm 10,000 bài đánh giá và cùng số epoch (5), mô hình `SelfAttentionClassifi` đạt **Accuracy** 76.33% và **F1** 76.69%, vượt xa các mô hình hồi quy ở LAB 05 (Simple RNN đạt $F_1 = 46.94\%$, LSTM đạt $F_1 = 51.39\%$, GRU đạt $F_1 = 66.76\%$).
- Thời gian huấn luyện của Self-Attention chỉ mất 8.10 **giây**, nhanh hơn đáng kể so với Simple RNN (13.47s) và GRU (32.75s). Khả năng xử lý song song toàn bộ ma trận thời gian là ưu thế cơ bản giúp loại bỏ vòng lặp tuần tự `for t in range(T)`.

1. **Vai trò sống còn của Scaling Factor ($\sqrt{d_k}$)**:

- Thực nghiệm tại Mục 14 xác nhận: Khi không có tỉ lệ, điểm số cực đại 4.0 đẩy softmax đạt tới 86.68% và làm teo tóp các vị trí khác xuống 1.59%. Khi chia cho $\sqrt{d_k} = 2$, phân phối trọng số mềm hóa (66.52% so với 9.00%), entropy tăng từ 0.449 lên 0.817, giữ cho gradient không bị bão hòa.

1. **Ảnh hưởng của Hệ số Nhiệt độ (Temperature)**:

- $T = 0.5$ khiến phân phối trở nên cực kỳ nhọn (Peaked), tiến sát hành vi của toán tử chọn cực đại arg max.
- $T = 2.0$ làm phẳng hóa phân phối (Entropy $= 1.0189$), triệt tiêu khả năng phân biệt từ trọng tâm.

1. **Bản đồ nhiệt Attention (Heatmap)**:

- Các cặp từ có liên hệ ngữ nghĩa trực tiếp (như `cat` ↔ `sleeping`, `book` ↔ `expensive`) nhận điểm tương tác cao hơn các từ đệm trung tính. Tuy nhiên, sự tập trung này mang tính tương quan thống kê chứ không chứng minh mạng nơ-ron ”hiểu” ngữ pháp theo nghĩa nhận thức của con người.

—

2. Bảng So Sánh Toàn Diện: LAB 05 (Recurrent) vs LAB 06 (Attention)

Tiêu chí so sánh (Property)	Mô hình hồi quy (RNN / LSTM / GRU — LAB 05)	Cơ chế Attention (Self-Attention — LAB 06)
Bảo toàn trật tự từ (Sequence Order)	Có sẵn tự nhiên nhờ bước lặp tuần tự $t = 1, 2, \ldots, T$.	**Không có sẵn**. Nếu không có Positional Encoding, Self-Attention có tính bất biến hoán vị (Permutation Equivariance).
Trạng thái ẩn nội tại (Hidden State)	Bắt buộc duy trì vector h_t nén lũy tích.	**Không bắt buộc**. Mỗi token được biểu diễn trực tiếp qua Query, Key, Value.
Tính toán tuần tự (Sequential Computation)	Bắt buộc tuần tự, không thể tính h_t khi chưa có h_{t-1}.	**Không bắt buộc**. Tính toán hoàn toàn song song qua phép nhân ma trận QK^T.
Tương tác trực tiếp giữa các token (Direct Interaction)	Hạn chế. Phải truyền qua $O(T)$ bước trung gian.	**Trực tiếp** 100%. Khoảng cách đường truyền tín hiệu giữa 2 token bất kỳ luôn là $O(1)$.
Phụ thuộc tầm xa (Long-range Dependency)	Rất khó khăn do triệt tiêu gradient và phân rã bộ nhớ.	**Rất dễ dàng**. Không có bước trễ thời gian.
Khả năng song song hóa (Parallelization)	Rất thấp trong quá trình huấn luyện chuỗi.	**Tối ưu tối đa** trên phần cứng GPU/TPU qua BLAS / Tensor Cores.
Chi phí theo độ dài chuỗi (Computational Cost)	Tuyến tính $O(T)$ về thời gian tính toán.	**Bậc hai** $O(T^2)$ về cả thời gian và bộ nhớ cho ma trận $T \times T$.
Biểu diễn ngữ cảnh (Context Representation)	Trạng thái ẩn cuối cùng h_T (Single compressed vector).	**Tổ hợp tuyến tính mềm** $c = \sum \alpha_i v_i$ cho từng truy vấn cụ thể.

—

3. Trả Lời Bộ Câu Hỏi Phỏng Vấn Học Phần (Individual Learning Check - Mục 31)

Câu 1: Attention score được tính như thế nào?

- **Trả lời**: Attention score s_i giữa một vector Query q và một vector Key k_i được tính bằng tích vô hướng (dot-product) thể hiện độ tương đồng hình học trong không gian vector:

$$s_i = q^T k_i = \sum_{d=1}^{d_k} q_d \cdot k_{id}$$

Tích vô hướng càng lớn nghĩa là hướng của Query và Key càng trùng nhau trong không gian biểu diễn, biểu thị độ liên quan ngữ nghĩa càng cao.

Câu 2: Tại sao phải dùng hàm Softmax?

- **Trả lời**: Softmax đảm nhiệm hai vai trò toán học quyết định:

1. **Chuẩn hóa xác suất**: Chuyển đổi các điểm số vô hướng có miền giá trị bất kỳ $(-\infty, +\infty)$ thành phân phối xác suất hợp lệ với $\alpha_i \in (0, 1)$ và tổng $\sum_i \alpha_i = 1$. Nhờ đó, context vector $c = \sum \alpha_i v_i$ trở thành một tổ hợp lồi (convex combination), bảo toàn tỉ lệ biên độ và không làm bùng nổ năng lượng của vector biểu diễn.
1. **Khuếch đại phi tuyến có đạo hàm mượt**: Hàm số mũ phóng đại sự khác biệt giữa các điểm số, cho phép mô hình dồn sự chú ý vào các token quan trọng nhất trong khi vẫn duy trì tính khả vi trên toàn miền để lan truyền ngược gradient.

Câu 3: Query, Key và Value có vai trò gì?

- **Trả lời**: Bộ ba (Q, K, V) bắt nguồn từ cơ chế truy vấn thông tin trong cơ sở dữ liệu:
- **Query (q)**: Đại diện cho thông tin hiện tại đang cần tìm kiếm hoặc vị trí token đang cần xây dựng biểu diễn ngữ cảnh (”Tôi đang tìm gì?”).
- **Key (k)**: Đại diện cho chỉ mục nội dung hoặc nhãn đặc trưng của từng token trong chuỗi (”Tôi có nội dung gì để đối khớp?”).
- **Value (v)**: Chứa đựng giá trị thông tin thực tế của token sẽ được trích xuất và tích lũy vào context vector nếu Key của nó khớp với Query (”Nội dung cụ thể của tôi là gì?”).

Câu 4: Context vector được tính như thế nào?

- **Trả lời**: Context vector c được tính bằng trung bình có trọng số (weighted sum) của toàn bộ các Value vectors v_i, với trọng số chính là các attention weights α_i nhận được từ Softmax:

$$c = \sum_{i=1}^{T} \alpha_i v_i = V^T \alpha$$

Câu 5: Tại sao scaled dot-product attention lại chia cho $\sqrt{d_k}$?

- **Trả lời**: Khi chiều không gian d_k của key tăng cao, nếu các thành phần của q và k là các biến ngẫu nhiên độc lập có trung bình bằng 0 và phương sai bằng 1, tích vô hướng $q^T k = \sum_{d=1}^{d_k} q_d k_d$ sẽ có phương sai bằng d_k và độ lệch chuẩn bằng $\sqrt{d_k}$.
- Khi d_k lớn, các giá trị tích vô hướng sẽ dao động với biên độ rất rộng. Đưa các giá trị quá lớn vào softmax sẽ đẩy phân phối vào vùng cực biên (bão hòa), nơi đạo hàm của hàm softmax tiệm cận 0, gây ra hiện tượng **triệt tiêu gradient (vanishing gradient)** nghiêm trọng trong quá trình huấn luyện. Phép chia cho $\sqrt{d_k}$ đưa phương sai trở lại mức 1, giúp ổn định động học gradient.

Câu 6: Self-attention khác attention thông thường ở đâu?

- **Trả lời**:
- Trong **Attention thông thường** (như trong bài toán dịch máy Seq2Seq Bahdanau/Luong): Query xuất phát từ một chuỗi độc lập (ví dụ trạng thái bộ giải mã Decoder), trong khi Key và Value đến từ chuỗi nguồn (bộ mã hóa Encoder) — gọi là Cross-Attention.
- Trong **Self-Attention**: Cả Query, Key và Value đều được sinh ra từ **chính cùng một chuỗi đầu vào** thông qua ba ma trận chiếu tuyến tính riêng biệt ($Q = XW_Q$, $K = XW_K$, $V = XW_V$). Self-attention cho phép các từ trong cùng một câu tự nhìn và tương tác với nhau để xây dựng biểu diễn ngữ cảnh nội tại.

Câu 7: Nếu sequence dài gấp đôi thì attention matrix thay đổi như thế nào?

- **Trả lời**: Kích thước của ma trận Attention là $T \times T$. Nếu độ dài chuỗi tăng gấp đôi ($T' = 2T$), số lượng phần tử của ma trận Attention sẽ tăng gấp 4 lần:

$$(2T)^2 = 4T^2$$

Chi phí lưu trữ bộ nhớ và số phép tính nhân ma trận QK^T đều tăng gấp 4 lần theo hàm bậc hai.

Câu 8: Attention có chứng minh mô hình đang reasoning (tư duy logic) không?

- **Trả lời**: **KHÔNG**. Attention weights phản ánh mức độ định tuyến thông tin số học dựa trên độ tương đồng không gian vector do tối ưu hóa hàm loss quy định. Trọng số attention cao chỉ là một điều kiện tương quan (correlation), không phải bằng chứng về quan hệ nhân quả (causation) hay lập luận logic ngữ nghĩa. Việc mô hình chú ý vào một từ không đồng nghĩa với việc mô hình hiểu được ý nghĩa triết học hay ngữ pháp của từ đó theo cách của con người.

Câu 9: Tại sao attention có thể xử lý long-range interaction tốt hơn recurrent state?

- **Trả lời**:
- Trong mô hình hồi quy (RNN/LSTM), để token ở vị trí $t = 1$ tương tác được với token ở vị trí $t = 100$, tín hiệu phải truyền tuần tự qua 99 phép biến đổi phi tuyến liên tiếp. Dù có cell state của LSTM, thông tin vẫn chịu suy giảm lũy thừa và gradient dễ bị phân rã.
- Trong Attention, khoảng cách tương tác giữa token 1 và token 100 luôn là 1 **bước trực tiếp** ($O(1)$ path length) thông qua tích vô hướng $q_1^T k_{100}$. Không có bất kỳ bước trung gian nào ở giữa, do đó thông tin tầm xa được bảo toàn nguyên vẹn 100%.

Câu 10: Attention giải quyết vấn đề gì của recurrent processing và tạo ra vấn đề mới nào?

- **Trả lời**:
- **Vấn đề giải quyết được**:

1. Xóa bỏ nút thắt cổ chai nén thông tin (Information Bottleneck) của hidden state.
1. Loại bỏ hoàn toàn đường truyền gradient dài qua thời gian, giải quyết triệt để bài toán phụ thuộc tầm xa (Long-range dependencies).
1. Cho phép tính toán song song hoàn toàn trên toàn bộ chuỗi, rút ngắn thời gian huấn luyện.

- **Vấn đề mới tạo ra**:

1. **Bùng nổ chi phí tính toán và bộ nhớ bậc hai** $O(T^2)$, khiến mô hình gặp khó khăn lớn khi xử lý văn bản rất dài (hàng nghìn đến hàng vạn token).
1. **Mất thông tin về trật tự từ**: Do phép tính QK^T và weighted sum độc lập với vị trí, Attention không biết từ nào đứng trước từ nào nếu không có cơ chế bù đắp tọa độ (Positional Encoding).

—

4. Mạch Nối Tiến Trình: LAB 05 ⟶ LAB 06 ⟶ LAB 07

Tiến trình ba bài thực hành liên tiếp minh họa một cách hoàn hảo dòng chảy lịch sử của ngành Xử lý ngôn ngữ tự nhiên hiện đại:

1. LAB 05: Recurrent State (Mô hình chuỗi tuần tự)

Chuỗi từ → RNN / LSTM / GRU → Trạng thái ẩn cuối cùng h_T → Phân loại cảm xúc

(Câu hỏi cốt lõi: Làm thế nào để duy trì thông tin qua các bước thời gian dài?)

2. LAB 06: Attention Mechanism (Cơ chế chú ý trọng tâm)

Chuỗi từ → Query, Key, Value → Scaled Dot-Product → Context Vector c → Phân loại

(Câu hỏi cốt lõi: Làm thế nào để chọn lọc thông tin liên quan trực tiếp giữa các vị trí?)

3. LAB 07: The Transformer Architecture (Kiến trúc Transformer hoàn chỉnh)

Self-Attention + Positional Encoding + Multi-Head Attention + Residual + LayerNorm

(Câu hỏi cốt lõi: Làm thế nào để xây dựng một kiến trúc hoàn chỉnh chỉ dựa trên Attention?)

Tại LAB 06, chúng ta đã nắm vững hạt nhân toán học cơ bản nhất: **tính Attention bằng tay và tự cài đặt tầng Scaled Dot-Product Attention từ đầu bằng NumPy**. Việc không đưa vội Multi-Head Attention hay Positional Encoding vào LAB 06 là một chủ đích sư phạm sâu sắc: Giúp người học nắm chắc bản chất đại số của từng phép nhân ma trận QK^T, cơ chế Softmax ổn định số học và tính toán context vector trước khi ghép chúng thành khối kiến trúc hoàn chỉnh của Transformer trong LAB 07.